

INDEX (MODULE 1)

SR. NO	(MODULE 1) TITLE	DATE	SIGNATURE
1.	1(A) Implement the Breadth First Search algorithm to solve a given problem. 1(B) Implement the Iterative Depth First Search algorithm to solve the same problem. 1(C) Compare the performance and efficiency of both algorithms.	28/07/2026	
2.	2(A) Implement the A* Search algorithm for solving a pathfinding problem. 2(B) Implement the Recursive Best-First Search algorithm for the same problem. 2(C) Compare the performance and effectiveness of both algorithms.	31/07/2026	
3.	3(A) Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset. 3(B) Evaluate the accuracy and effectiveness of the decision tree on test data. 3(C) Visualize and interpret the generated decision tree.	04/08/2026	
4.	4(A) Implement the Feed Forward Backpropagation algorithm to train a neural network. 4(B) Use a given dataset to train the neural network for a specific task. 4(C) Evaluate the performance of the trained network on test data.	06/08/2026	
5.	5(A) Implement the SVM algorithm for binary classification. 5(B) Train an SVM model using a given dataset and optimize its parameters. 5(C) Evaluate the performance of the SVM model on test data and analyze the results.	07/08/2026	
6.	6(A) Implement the Adaboost algorithm to create an ensemble of weak classifiers. 6(B) Train the ensemble model on a given dataset and evaluate its performance. 6(C) Compare the results with individual weak classifiers.	10/08/2026	
7.	7(A) Implement the Naive Bayes' algorithm for classification. 7(B) Train a Naive Bayes' model using a given dataset and calculate class probabilities. 7(C) Evaluate the accuracy of the model on test data and analyze the results.	11/08/2026	
8.	8(A) Implement the K-NN algorithm for classification or regression. 8(B) Apply the K-NN algorithm to a given dataset and predict the class or value for test data. 8(C) Evaluate the accuracy or error of the predictions and analyze the results.	12/08/2026	
9.	9(A) Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent itemsets. 9(B) Generate association rules from the frequent itemsets and calculate their support and confidence. 9(C) Interpret and analyze the discovered association rules.	14/08/2026	

PRACTICAL NO 1 (A)

AIM: Implement the Breadth First Search algorithm to solve a given problem.

CODE:

```
from collections import deque
```

```
def bfs(graph, start_node):
```

```
    """
```

```
    Performs BFS traversal on an adjacency list graph representation.
```

```
    :param graph: dict mapping node -> list of neighboring nodes
```

```
    :param start_node: the node to begin traversal from
```

```
    :return: list of nodes in the order they were visited
```

```
    """
```

```
    visited = set()
```

```
    queue = deque([start_node])
```

```
    # Mark the initial node as visited
```

```
    visited.add(start_node)
```

```
    traversal_order = []
```

```
    while queue:
```

```
        # Pop the first element added to the queue (FIFO)
```

```
        current = queue.popleft()
```

```
        traversal_order.append(current)
```

```

        # Explore all neighbors

        for neighbor in graph.get(current, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return traversal_order

# --- Example Usage ---

if __name__ == "__main__":
    # Graph represented as an adjacency list
    #      A
    #      / \
    #     B   C
    #    / \   \
    #   D   E   F

    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': ['F'],
        'F': []
    }

```

```
}
```

```
start = 'A'
```

```
result = bfs(graph, start)
```

```
print(f"BFS Order starting from '{start}':", " -> ".join(result))
```

OUTPUT:

```
===== RESTART: C:/SANJANA TYCS/PYTHON_ALL_PRACT  
BFS Order starting from 'A': A -> B -> C -> D -> E -> F
```

PRACTICAL NO 1(B)

AIM: Implement the Iterative Depth First Search algorithm to solve the same problem.

CODE:

```
def dfs_iterative(graph, start_node):
```

```
    """
```

Performs Iterative Depth First Search traversal on an adjacency list graph representation.

```
    :param graph: dict mapping node -> list of neighboring nodes
```

```
    :param start_node: the node to begin traversal from
```

```
    :return: list of nodes in the order they were visited
```

```
    """
```

```
    visited = set()
```

```
    stack = [start_node]    # Use a stack instead of a queue
```

```
    traversal_order = []
```

```
    while stack:
```

```
        # Pop the last element added (LIFO)
```

```
        current = stack.pop()
```

```
        if current not in visited:
```

```
            visited.add(current)
```

```
            traversal_order.append(current)
```

```
            # Add neighbors to stack
```

```

        # Reverse ensures left-to-right traversal order
        for neighbor in reversed(graph.get(current, [])):
            if neighbor not in visited:
                stack.append(neighbor)

    return traversal_order

# --- Example Usage ---
if __name__ == "__main__":
    # Graph represented as an adjacency list
    #
    #   A
    #  / \
    # B   C
    # / \  \
    # D  E  F
    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': ['F'],
        'F': []
    }

```

```
start = 'A'

result = dfs_iterative(graph, start)

print(f"DFS Order starting from '{start}':", " -> ".join(result))
```

OUTPUT:

```
===== RESTART: C:/SANJANA TYCS/PYTHON_ALL_PRACTICAL/plb.py
DFS Order starting from 'A': A -> B -> D -> E -> F -> C
```

PRACTICAL NO 1 (C)

AIM: Compare the performance and efficiency of both algorithms.

CODE:

```
import time

from collections import deque

def bfs(graph, start_node):
    visited = set()
    queue = deque([start_node])
    visited.add(start_node)
    traversal_order = []

    while queue:
        current = queue.popleft()
        traversal_order.append(current)
        for neighbor in graph.get(current, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return traversal_order

def dfs_iterative(graph, start_node):
    visited = set()
```

```

stack = [start_node]
traversal_order = []

while stack:
    current = stack.pop()
    if current not in visited:
        visited.add(current)
        traversal_order.append(current)
        for neighbor in reversed(graph.get(current, [])):
            if neighbor not in visited:
                stack.append(neighbor)
return traversal_order

# --- Example Usage ---
if __name__ == "__main__":
    # Graph represented as an adjacency list
    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': ['F'],
        'F': []
    }

```

```

start = 'A'

# Measure BFS performance
bfs_start = time.time()
bfs_result = bfs(graph, start)
bfs_end = time.time()

# Measure DFS performance
dfs_start = time.time()
dfs_result = dfs_iterative(graph, start)
dfs_end = time.time()

# Print results
print("BFS Traversal:", " -> ".join(bfs_result))
print("DFS Traversal:", " -> ".join(dfs_result))
print(f"BFS Execution Time: {bfs_end - bfs_start:.6f} seconds")
print(f"DFS Execution Time: {dfs_end - dfs_start:.6f} seconds")

```

OUTPUT:

```

===== RESTART: C:/SANJANA TYCS/PYT
BFS Traversal: A -> B -> C -> D -> E -> F
DFS Traversal: A -> B -> D -> E -> F -> C
BFS Execution Time: 0.000016 seconds
DFS Execution Time: 0.000006 seconds

```

PRACTICAL NO 2(A)

AIM: Implement the A* Search algorithm for solving a pathfinding problem.

CODE:

```
import heapq

class Node:

    def __init__(self, position, parent=None):

        self.position = position

        self.parent = parent

        self.g = 0 # Cost from start to current node

        self.h = 0 # Heuristic cost to goal

        self.f = 0 # Total cost


    def __lt__(self, other):

        return self.f < other.f


def heuristic(a, b):

    # Manhattan distance

    return abs(a[0] - b[0]) + abs(a[1] - b[1])


def astar_search(grid, start, end):

    open_list = []

    closed_set = set()

    start_node = Node(start)
```

```

end_node = Node(end)

heapq.heappush(open_list, start_node)

while open_list:
    current_node = heapq.heappop(open_list)
    closed_set.add(current_node.position)

    # Goal check
    if current_node.position == end_node.position:
        path = []
        while current_node:
            path.append(current_node.position)
            current_node = current_node.parent
        return path[::-1] # Reverse path

    # Explore neighbors (up, down, left, right)
    neighbors = [(0,1),(0,-1),(1,0),(-1,0)]
    for dx, dy in neighbors:
        neighbor_pos = (current_node.position[0] + dx, current_node.position[1] +
dy)

        # Check boundaries and obstacles
        if (0 <= neighbor_pos[0] < len(grid) and
            0 <= neighbor_pos[1] < len(grid[0]) and

```

```

        grid[neighbor_pos[0]][neighbor_pos[1]] == 0 and
        neighbor_pos not in closed_set):

    neighbor_node = Node(neighbor_pos, current_node)
    neighbor_node.g = current_node.g + 1
    neighbor_node.h = heuristic(neighbor_pos, end_node.position)
    neighbor_node.f = neighbor_node.g + neighbor_node.h

    # Check if already in open list with lower cost
    if any(open_node.position == neighbor_node.position and open_node.g
    <= neighbor_node.g for open_node in open_list):
        continue

    heapq.heappush(open_list, neighbor_node)

return None # No path found

# --- Example Usage ---
if __name__ == "__main__":
    grid = [
        [0, 1, 0, 0, 0],
        [0, 1, 0, 1, 0],
        [0, 0, 0, 1, 0],
        [0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0]

```

```
]
```

```
start = (0, 0)
```

```
end = (4, 4)
```

```
path = astar_search(grid, start, end)
```

```
print("Shortest Path:", path)
```

OUTPUT:

```
===== RESTART: C:/SANJANA TYCS/PYTHON ALL PRACTICAL/p2a.py =====  
Shortest Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3),  
                (4, 4)]
```

PRACTICAL 2(B)

AIM: Implement the Recursive Best-First Search algorithm for the same problem.

CODE:

```
import math

class Node:

    def __init__(self, position, parent=None, g=0, h=0):

        self.position = position

        self.parent = parent

        self.g = g # Cost from start

        self.h = h # Heuristic cost to goal

        self.f = g + h # Total cost


def heuristic(a, b):

    # Manhattan distance

    return abs(a[0] - b[0]) + abs(a[1] - b[1])


def get_neighbors(position, grid):

    neighbors = []

    moves = [(0,1),(0,-1),(1,0),(-1,0)]

    for dx, dy in moves:

        nx, ny = position[0] + dx, position[1] + dy

        if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]) and grid[nx][ny] == 0:

            neighbors.append((nx, ny))
```

```

    return neighbors

def rbfs(node, goal, grid, f_limit):
    if node.position == goal:
        path = []
        while node:
            path.append(node.position)
            node = node.parent
        return path[::-1], True, 0

    successors = []
    for pos in get_neighbors(node.position, grid):
        g = node.g + 1
        h = heuristic(pos, goal)
        successor = Node(pos, node, g, h)
        successors.append(successor)

    if not successors:
        return None, False, math.inf

    for s in successors:
        s.f = max(s.g + s.h, node.f)

    while True:
        successors.sort(key=lambda x: x.f)

```

```

    best = successors[0]

    if best.f > f_limit:

        return None, False, best.f

    alternative = successors[1].f if len(successors) > 1 else math.inf

    result, found, best.f = rbfs(best, goal, grid, min(f_limit, alternative))

    if found:

        return result, True, best.f

# --- Example Usage ---

if __name__ == "__main__":

    grid = [

        [0, 1, 0, 0, 0],

        [0, 1, 0, 1, 0],

        [0, 0, 0, 1, 0],

        [0, 1, 1, 1, 0],

        [0, 0, 0, 0, 0]

    ]

    start = (0, 0)

    goal = (4, 4)

    start_node = Node(start, None, 0, heuristic(start, goal))

    path, found, _ = rbfs(start_node, goal, grid, math.inf)

    if found:

```

```
        print("Path found:", path)

    else:

        print("No path found")
```

OUTPUT:

```
----- RESTART: C:/SANGHANA ITCS/PYTHON_APP_PRACTICAL/p20.py -----
Path found: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4,
4)]
```

PRACTICAL 2(C)

AIM: Compare the performance and effectiveness of both algorithms.

CODE:

```
import heapq, math, time

# ----- Common Utilities -----

class Node:

    def __init__(self, position, parent=None, g=0, h=0):

        self.position = position

        self.parent = parent

        self.g = g

        self.h = h

        self.f = g + h

    def __lt__(self, other):

        return self.f < other.f

def heuristic(a, b):

    return abs(a[0] - b[0]) + abs(a[1] - b[1]) # Manhattan distance

def get_neighbors(position, grid):

    moves = [(0,1),(0,-1),(1,0),(-1,0)]

    neighbors = []

    for dx, dy in moves:
```

```

        nx, ny = position[0] + dx, position[1] + dy

        if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]) and grid[nx][ny] == 0:
            neighbors.append((nx, ny))

    return neighbors

# ----- A* Search -----
def astar_search(grid, start, end):
    open_list = []
    closed_set = set()
    start_node = Node(start, None, 0, heuristic(start, end))
    heapq.heappush(open_list, start_node)
    nodes_expanded = 0

    while open_list:
        current = heapq.heappop(open_list)
        nodes_expanded += 1
        closed_set.add(current.position)

        if current.position == end:
            path = []
            while current:
                path.append(current.position)
                current = current.parent
            return path[::-1], nodes_expanded

```

```

        for pos in get_neighbors(current.position, grid):
            if pos in closed_set:
                continue

            g = current.g + 1
            h = heuristic(pos, end)
            neighbor = Node(pos, current, g, h)

            if any(n.position == neighbor.position and n.g <= neighbor.g for n in
open_list):
                continue

            heapq.heappush(open_list, neighbor)

    return None, nodes_expanded

# ----- RBFS -----
def rbfs(node, goal, grid, f_limit, nodes_expanded=0):
    nodes_expanded += 1
    if node.position == goal:
        path = []
        while node:
            path.append(node.position)
            node = node.parent
        return path[::-1], True, 0, nodes_expanded

    successors = []
    for pos in get_neighbors(node.position, grid):

```

```

    g = node.g + 1
    h = heuristic(pos, goal)
    successors.append(Node(pos, node, g, h))

if not successors:
    return None, False, math.inf, nodes_expanded

for s in successors:
    s.f = max(s.g + s.h, node.f)

while True:
    successors.sort(key=lambda x: x.f)
    best = successors[0]
    if best.f > f_limit:
        return None, False, best.f, nodes_expanded
    alternative = successors[1].f if len(successors) > 1 else math.inf
    result, found, best.f, nodes_expanded = rbfs(best, goal, grid, min(f_limit,
alternative), nodes_expanded)
    if found:
        return result, True, best.f, nodes_expanded

# ----- Example Usage -----
if __name__ == "__main__":
    grid = [
        [0, 1, 0, 0, 0],

```

```

        [0, 1, 0, 1, 0],
        [0, 0, 0, 1, 0],
        [0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0]
    ]

    start, goal = (0,0), (4,4)

    # Run A*

    t1 = time.time()

    path_astar, expanded_astar = astar_search(grid, start, goal)

    t2 = time.time()

    # Run RBFS

    t3 = time.time()

    start_node = Node(start, None, 0, heuristic(start, goal))

    path_rbfs, found, _, expanded_rbfs = rbfs(start_node, goal, grid, math.inf)

    t4 = time.time()

    # Results

    print("A* Path:", path_astar)

    print("A* Nodes Expanded:", expanded_astar)

    print("A* Time:", round(t2-t1, 6), "seconds\n")

    print("RBFS Path:", path_rbfs)

    print("RBFS Nodes Expanded:", expanded_rbfs)

```

```
print("RBFS Time:", round(t4-t3, 6), "seconds")
```

OUTPUT:

```
===== RESTART: C:/SANJANA TYCS/PYTHON_ALL_PRACTICAL/p2c.py =====  
A* Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)  
]  
A* Nodes Expanded: 11  
A* Time: 5.5e-05 seconds  
  
RBFS Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4,  
4)]  
RBFS Nodes Expanded: 11  
RBFS Time: 3.7e-05 seconds  
|
```

Practical 3 (A)

AIM: Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset

CODE:

```
import pandas as pd
import numpy as np

# Sample dataset: Play Tennis
data = {
    'Outlook':
['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunny','Sunny','Rain','Sunny','Overcast','Overcast','Rain'],
    'Temperature':
['Hot','Hot','Hot','Mild','Cool','Cool','Mild','Hot','Cool','Mild','Mild','Mild','Hot','Mild'],
    'Humidity':
['High','High','High','High','Normal','Normal','Normal','High','Normal','Normal','Normal','High','Normal','High'],
    'Wind':
['Weak','Strong','Weak','Weak','Weak','Strong','Strong','Weak','Weak','Weak','Strong','Strong','Weak','Strong'],
    'PlayTennis': ['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes','No']
}

df = pd.DataFrame(data)
```

```

# Entropy calculation
def entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    entropy_val = 0
    for i in range(len(elements)):
        prob = counts[i]/np.sum(counts)
        entropy_val += -prob*np.log2(prob)
    return entropy_val

# Information Gain
def info_gain(data, split_attribute, target_name="PlayTennis"):
    total_entropy = entropy(data[target_name])
    vals, counts = np.unique(data[split_attribute], return_counts=True)
    weighted_entropy = 0
    for i in range(len(vals)):
        subset = data[data[split_attribute] == vals[i]]
        weighted_entropy += (counts[i]/np.sum(counts)) * entropy(subset[target_name])
    return total_entropy - weighted_entropy

# ID3 Algorithm
def id3(data, original_data, features, target_attribute="PlayTennis",
parent_node_class=None):
    # If all target values are the same → return that value
    if len(np.unique(data[target_attribute])) <= 1:

```

```

        return np.unique(data[target_attribute])[0]

# If dataset empty → return mode of original dataset
elif len(data) == 0:

    return np.unique(original_data[target_attribute])
[np.argmax(np.unique(original_data[target_attribute], return_counts=True)[1])]

# If no features left → return parent node class
elif len(features) == 0:

    return parent_node_class

else:

    parent_node_class = np.unique(data[target_attribute])
[np.argmax(np.unique(data[target_attribute], return_counts=True)[1])]

    # Choose best feature

    gains = [info_gain(data, feature, target_attribute) for feature in features]

    best_feature = features[np.argmax(gains)]

    tree = {best_feature:{}}

    # Grow tree

    for value in np.unique(data[best_feature]):

        sub_data = data[data[best_feature] == value]

        subtree = id3(sub_data, df, [f for f in features if f != best_feature],
target_attribute, parent_node_class)

        tree[best_feature][value] = subtree

    return tree


# Build decision tree

features = list(df.columns[:-1])

decision_tree = id3(df, df, features)

```

```
print("Decision Tree:\n", decision_tree)
```

OUTPUT:

```
===== RESTART: C:/SANJANA SYCS/p3aaa.py =====  
Decision Tree:  
{ 'Outlook': { 'Overcast': 'Yes', 'Rain': { 'Wind': { 'Strong': 'No', 'Weak': 'Yes' } }, 'Sunny'  
: { 'Temperature': { 'Cool': 'Yes', 'Hot': 'No', 'Mild': 'Yes' } } } }  
>|
```

PRACTICAL 3(B)

AIM: Evaluate the accuracy and effectiveness of the decision tree on test data.

CODE:

```
# Step 1: Import libraries
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
# Step 2: Load dataset (Iris dataset as example)
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Step 3: Split into train and test sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42
```

```
)
```

```
# Step 4: Train Decision Tree
```

```
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
```

Step 5: Predict on test data

```
y_pred = clf.predict(X_test)
```

Step 6: Evaluate accuracy and effectiveness

```
print("✅ Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\n📊 Classification Report:\n", classification_report(y_test, y_pred))
```

```
print("\n🔍 Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

OUTPUT:

```

REPORT: 0.75 accuracy & 1.00 per
✅ Accuracy: 1.0

📊 Classification Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00        19
     1       1.00      1.00      1.00        13
     2       1.00      1.00      1.00        13

   accuracy                   1.00           45
  macro avg              1.00      1.00      1.00           45
 weighted avg              1.00      1.00      1.00           45

🔍 Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

```

PRACTICAL 3(C)

AIM: Visualize and interpret the generated decision tree.

CODE:

```
# Step 1: Import libraries

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

import matplotlib.pyplot as plt


# Step 2: Load dataset (Iris dataset as example)

data = load_iris()

X = data.data

y = data.target


# Step 3: Split into train and test sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 4: Train Decision Tree

clf = DecisionTreeClassifier(random_state=42, max_depth=3) # limit depth for clarity

clf.fit(X_train, y_train)


# Step 5: Visualize the tree

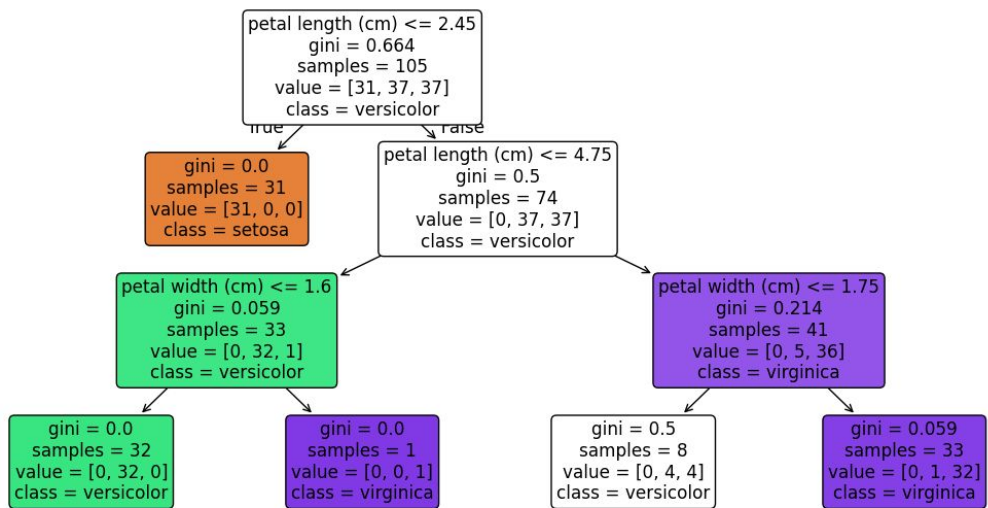
plt.figure(figsize=(12,8))
```

```

plot_tree(
    clf,
    feature_names=data.feature_names,
    class_names=data.target_names,
    filled=True,
    rounded=True
)
plt.show()

```

OUTPUT:



PRACTICAL 4(A)

AIM: Implement the Feed Forward Backpropagation algorithm to train a neural network.

CODE:

```
import numpy as np

# Activation function (Sigmoid) and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Training data (XOR problem as example)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

y = np.array([[0],
              [1],
              [1],
              [0]])

# Initialize weights randomly
np.random.seed(42)
```

```

inputLayerNeurons = 2
hiddenLayerNeurons = 2
outputLayerNeurons = 1

# Weight & bias initialization
wh = np.random.uniform(size=(inputLayerNeurons, hiddenLayerNeurons))
bh = np.random.uniform(size=(1, hiddenLayerNeurons))
wo = np.random.uniform(size=(hiddenLayerNeurons, outputLayerNeurons))
bo = np.random.uniform(size=(1, outputLayerNeurons))

# Training process
lr = 0.1
epochs = 10000

for i in range(epochs):
    # Forward Propagation
    hidden_input = np.dot(X, wh) + bh
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, wo) + bo
    final_output = sigmoid(final_input)

    # Backpropagation
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)

```

```

error_hidden = d_output.dot(wo.T)

d_hidden = error_hidden * sigmoid_derivative(hidden_output)

# Update weights and biases

wo += hidden_output.T.dot(d_output) * lr
bo += np.sum(d_output, axis=0, keepdims=True) * lr
wh += X.T.dot(d_hidden) * lr
bh += np.sum(d_hidden, axis=0, keepdims=True) * lr

# Final output after training
print("Final Output after Training:\n", final_output)

```

OUTPUT:

```

Final Output after Training:
[[0.05322146]
 [0.95171535]
 [0.95160449]
 [0.05175396]]

```

PRACTICAL 4(B)

AIM: Use a given dataset to train the neural network for a specific task.

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import OneHotEncoder


# Activation function (Sigmoid) and derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)


# Load dataset
iris = load_iris()

X = iris.data    # features: sepal length, sepal width, petal length, petal width
y = iris.target.reshape(-1,1)


# One-hot encode target labels (Setosa, Versicolor, Virginica)
encoder = OneHotEncoder(sparse_output=False)
y = encoder.fit_transform(y)


# Train-test split
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Network architecture

input_neurons = X_train.shape[1] # 4 features
hidden_neurons = 6 # hidden layer size
output_neurons = y_train.shape[1] # 3 classes

# Initialize weights and biases

np.random.seed(42)

wh = np.random.uniform(size=(input_neurons, hidden_neurons))
bh = np.random.uniform(size=(1, hidden_neurons))
wo = np.random.uniform(size=(hidden_neurons, output_neurons))
bo = np.random.uniform(size=(1, output_neurons))

# Training parameters

lr = 0.1
epochs = 5000

# Training loop
for i in range(epochs):
    # Forward pass
    hidden_input = np.dot(X_train, wh) + bh
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, wo) + bo

```

```

final_output = sigmoid(final_input)

# Backpropagation
error = y_train - final_output
d_output = error * sigmoid_derivative(final_output)

error_hidden = d_output.dot(wo.T)
d_hidden = error_hidden * sigmoid_derivative(hidden_output)

# Update weights
wo += hidden_output.T.dot(d_output) * lr
bo += np.sum(d_output, axis=0, keepdims=True) * lr
wh += X_train.T.dot(d_hidden) * lr
bh += np.sum(d_hidden, axis=0, keepdims=True) * lr

# Test the network
hidden_input_test = np.dot(X_test, wh) + bh
hidden_output_test = sigmoid(hidden_input_test)

final_input_test = np.dot(hidden_output_test, wo) + bo
final_output_test = sigmoid(final_input_test)

# Predictions
predictions = np.argmax(final_output_test, axis=1)
true_labels = np.argmax(y_test, axis=1)

```

```
accuracy = np.mean(predictions == true_labels)
```

```
print("✅ Test Accuracy:", accuracy)
```

```
print("🔍 Predictions:", predictions[:10])
```

```
print("🎯 True Labels:", true_labels[:10])
```

OUTPUT:

```
-----  
✅ Test Accuracy: 0.7111111111111111  
🔍 Predictions: [2 0 2 2 2 0 2 2 2 2]  
🎯 True Labels: [1 0 2 1 1 0 1 2 1 1]  
.
```

PRACTICAL 4(C)

AIM: Evaluate the performance of the trained network on test data.

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import OneHotEncoder

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Activation function (Sigmoid) and derivative
def sigmoid(x):

    return 1 / (1 + np.exp(-x))


def sigmoid_derivative(x):

    return x * (1 - x)


# Load dataset

iris = load_iris()

X = iris.data

y = iris.target.reshape(-1,1)


# One-hot encode target labels

encoder = OneHotEncoder(sparse_output=False)

y = encoder.fit_transform(y)
```

```

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Network architecture
input_neurons = X_train.shape[1]    # 4 features
hidden_neurons = 6                   # hidden layer size
output_neurons = y_train.shape[1]    # 3 classes

# Initialize weights and biases
np.random.seed(42)
wh = np.random.uniform(size=(input_neurons, hidden_neurons))
bh = np.random.uniform(size=(1, hidden_neurons))
wo = np.random.uniform(size=(hidden_neurons, output_neurons))
bo = np.random.uniform(size=(1, output_neurons))

# Training parameters
lr = 0.1
epochs = 5000

# Training loop
for i in range(epochs):
    # Forward pass
    hidden_input = np.dot(X_train, wh) + bh
    hidden_output = sigmoid(hidden_input)

```

```

final_input = np.dot(hidden_output, wo) + bo
final_output = sigmoid(final_input)

# Backpropagation
error = y_train - final_output
d_output = error * sigmoid_derivative(final_output)

error_hidden = d_output.dot(wo.T)
d_hidden = error_hidden * sigmoid_derivative(hidden_output)

# Update weights
wo += hidden_output.T.dot(d_output) * lr
bo += np.sum(d_output, axis=0, keepdims=True) * lr
wh += X_train.T.dot(d_hidden) * lr
bh += np.sum(d_hidden, axis=0, keepdims=True) * lr

# ---- Evaluation on Test Data ----
hidden_input_test = np.dot(X_test, wh) + bh
hidden_output_test = sigmoid(hidden_input_test)

final_input_test = np.dot(hidden_output_test, wo) + bo
final_output_test = sigmoid(final_input_test)

# Predictions
predictions = np.argmax(final_output_test, axis=1)

```

```
true_labels = np.argmax(y_test, axis=1)
```

```
# Performance metrics (fixed with zero_division=0)
```

```
print("✅ Test Accuracy:", accuracy_score(true_labels, predictions))
```

```
print("\n📊 Classification Report:\n", classification_report(true_labels, predictions,  
zero_division=0))
```

```
print("\n🔍 Confusion Matrix:\n", confusion_matrix(true_labels, predictions))
```

OUTPUT:

```
✅ Test Accuracy: 0.7111111111111111

📊 Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00        19
     1           0.00        0.00        0.00        13
     2           0.50        1.00        0.67        13

   accuracy                   0.71         45
  macro avg           0.50        0.67        0.56         45
 weighted avg           0.57        0.71        0.61         45

🔍 Confusion Matrix:
[[19  0  0]
 [ 0  0 13]
 [ 0  0 13]]
```

PRACTICAL 5(A)

AIM: Implement the SVM algorithm for binary classification.

CODE:

```
# Step 1: Import libraries

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Step 2: Load dataset (Breast Cancer dataset)

data = datasets.load_breast_cancer()

X = data.data

y = data.target    # Binary labels: 0 = malignant, 1 = benign


# Step 3: Split into train and test sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 4: Train SVM model

svm_model = SVC(kernel='linear', random_state=42)

svm_model.fit(X_train, y_train)


# Step 5: Predict on test data

y_pred = svm_model.predict(X_test)
```

Step 6: Evaluate performance

```
print("✅ Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\n📊 Classification Report:\n", classification_report(y_test, y_pred, zero_division=0))
```

```
print("\n🔗 Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

OUTPUT:

```
✅ Accuracy: 0.9649122807017544
```

```
📊 Classification Report:
```

	precision	recall	f1-score	support
0	0.97	0.94	0.95	63
1	0.96	0.98	0.97	108
accuracy			0.96	171
macro avg	0.97	0.96	0.96	171
weighted avg	0.96	0.96	0.96	171

```
🔗 Confusion Matrix:
```

```
[[ 59   4]
 [  2 106]]
```

PRACTICAL 5(B)

AIM: Train an SVM model using a given dataset and optimize its parameters.

CODE:

```
# Step 1: Import libraries

from sklearn import datasets

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Step 2: Load dataset (Breast Cancer dataset for binary classification)

data = datasets.load_breast_cancer()

X = data.data

y = data.target    # Binary labels: 0 = malignant, 1 = benign


# Step 3: Split into train and test sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 4: Define SVM model and parameter grid

svm = SVC()

param_grid = {
    'C': [0.1, 1, 10],          # keep grid small
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
```

```
}
```

```
# Step 5: Perform Grid Search with Cross-Validation
```

```
# IMPORTANT: set n_jobs=1 to avoid worker crashes on Windows
```

```
grid_search = GridSearchCV(
```

```
    estimator=svm,
```

```
    param_grid=param_grid,
```

```
    cv=5,
```

```
    scoring='accuracy',
```

```
    verbose=1,
```

```
    n_jobs=1
```

```
)
```

```
grid_search.fit(X_train, y_train)
```

```
# Step 6: Best parameters and model
```

```
print("✅ Best Parameters:", grid_search.best_params_)
```

```
best_model = grid_search.best_estimator_
```

```
# Step 7: Evaluate on test data
```

```
y_pred = best_model.predict(X_test)
```

```
print("✅ Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\n📊 Classification Report:\n", classification_report(y_test, y_pred, zero_division=0))
```

```
print("\n🔗 Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

OUTPUT:

Fitting 5 folds for each of 12 candidates, totalling 60 fits

✓ Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}

✓ Test Accuracy: 0.9707602339181286

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.97	0.96	63
1	0.98	0.97	0.98	108
accuracy			0.97	171
macro avg	0.97	0.97	0.97	171
weighted avg	0.97	0.97	0.97	171

Confusion Matrix:

```
[[ 61   2]
 [   3 105]]
```

PRACTICAL 5(C)

AIM: Evaluate the performance of the SVM model on test data and analyze the results.

CODE:

```
# Step 1: Import libraries

from sklearn import datasets

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Step 2: Load dataset (Breast Cancer dataset for binary classification)

data = datasets.load_breast_cancer()

X = data.data

y = data.target    # Binary labels: 0 = malignant, 1 = benign


# Step 3: Split into train and test sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 4: Train optimized SVM model (best parameters found earlier)

svm_model = SVC(C=10, kernel='rbf', gamma='scale', random_state=42)

svm_model.fit(X_train, y_train)


# Step 5: Predict on test data

y_pred = svm_model.predict(X_test)
```

Step 6: Evaluate performance

```
print("✅ Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\n📊 Classification Report:\n", classification_report(y_test, y_pred, zero_division=0))
```

```
print("\n🔑 Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

OUTPUT:

```
✅ Test Accuracy: 0.9415204678362573
```

```
📊 Classification Report:
```

	precision	recall	f1-score	support
0	0.98	0.86	0.92	63
1	0.92	0.99	0.96	108
accuracy			0.94	171
macro avg	0.95	0.92	0.94	171
weighted avg	0.94	0.94	0.94	171

```
🔑 Confusion Matrix:
```

```
[[ 54   9]
 [   1 107]]
```

PRACTICAL 6(A)

AIM: Implement the Adaboost algorithm to create an ensemble of weak classifiers.

CODE:

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.datasets import make_classification

# Create dataset
X, y = make_classification(n_samples=500, n_features=10,
                           n_informative=5, n_redundant=0,
                           random_state=42)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.3,
                                                    random_state=42)

# Weak classifier: Decision Stump
weak_clf = DecisionTreeClassifier(max_depth=1)

# ☒ Use 'estimator' instead of 'base_estimator'
ada_clf = AdaBoostClassifier(estimator=weak_clf,
```

```
n_estimators=50,  
learning_rate=1.0,  
random_state=42)  
  
# Train  
ada_clf.fit(X_train, y_train)  
  
# Predictions  
y_pred_train = ada_clf.predict(X_train)  
y_pred_test = ada_clf.predict(X_test)  
  
# Accuracy  
train_acc = accuracy_score(y_train, y_pred_train)  
test_acc = accuracy_score(y_test, y_pred_test)  
  
print("Training Accuracy:", train_acc)  
print("Testing Accuracy:", test_acc)
```

OUTPUT:

```
Training Accuracy: 0.9142857142857143  
Testing Accuracy: 0.8733333333333333
```

PRACTICAL 6(B)

AIM: Train the ensemble model on a given dataset and evaluate its performance.

CODE:

```
import numpy as np

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report


# Step 1: Load dataset
data = load_breast_cancer()
X, y = data.data, data.target


# Step 2: Split into train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 3: Define weak classifier (Decision Stump)
weak_clf = DecisionTreeClassifier(max_depth=1)


# Step 4: Build AdaBoost ensemble
ada_clf = AdaBoostClassifier()
```

```

    estimator=weak_clf,    # use 'estimator' for new sklearn versions
    n_estimators=100,
    learning_rate=1.0,
    random_state=42
)

# Step 5: Train model
ada_clf.fit(X_train, y_train)

# Step 6: Evaluate
y_pred_train = ada_clf.predict(X_train)
y_pred_test  = ada_clf.predict(X_test)

print("Training Accuracy:", accuracy_score(y_train, y_pred_train))
print("Testing Accuracy:", accuracy_score(y_test, y_pred_test))
print("\nClassification Report:\n", classification_report(y_test, y_pred_test))

```

OUTPUT:

```

Training Accuracy: 1.0
Testing Accuracy: 0.9707602339181286

Classification Report:
      precision    recall  f1-score   support

     0       0.94       0.98       0.96         63
     1       0.99       0.96       0.98        108

 accuracy          0.97         171
 macro avg       0.96       0.97       0.97         171
weighted avg       0.97       0.97       0.97         171

```

PRACTICAL 6(C)

AIM: Compare the results with individual weak classifiers.

CODE:

```
import numpy as np

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset

data = load_breast_cancer()

X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Weak classifier (Decision Stump)

weak_clf = DecisionTreeClassifier(max_depth=1)

weak_clf.fit(X_train, y_train)

weak_train_acc = accuracy_score(y_train, weak_clf.predict(X_train))

weak_test_acc = accuracy_score(y_test, weak_clf.predict(X_test))

# AdaBoost ensemble

ada_clf = AdaBoostClassifier(estimator=weak_clf, n_estimators=100, learning_rate=1.0,
random_state=42)

ada_clf.fit(X_train, y_train)
```

```
ada_train_acc = accuracy_score(y_train, ada_clf.predict(X_train))  
ada_test_acc = accuracy_score(y_test, ada_clf.predict(X_test))
```

```
print("Weak Classifier Training Accuracy:", weak_train_acc)  
print("Weak Classifier Testing Accuracy:", weak_test_acc)  
print("AdaBoost Training Accuracy:", ada_train_acc)  
print("AdaBoost Testing Accuracy:", ada_test_acc)
```

OUTPUT:

```
| Weak Classifier Training Accuracy: 0.9246231155778895  
| Weak Classifier Testing Accuracy: 0.8947368421052632  
| AdaBoost Training Accuracy: 1.0  
| AdaBoost Testing Accuracy: 0.9707602339181286  
.,|
```

PRACTICAL 7(A)

AIM: Implement the Naive Bayes' algorithm for classification.

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report


# Step 1: Load dataset

iris = load_iris()

X, y = iris.data, iris.target


# Step 2: Split into train/test

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 3: Initialize Naive Bayes classifier

nb_clf = GaussianNB()


# Step 4: Train model

nb_clf.fit(X_train, y_train)


# Step 5: Predict
```

```
y_pred_train = nb_clf.predict(X_train)
```

```
y_pred_test = nb_clf.predict(X_test)
```

```
# Step 6: Evaluate
```

```
print("Training Accuracy:", accuracy_score(y_train, y_pred_train))
```

```
print("Testing Accuracy:", accuracy_score(y_test, y_pred_test))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_test))
```

OUTPUT:

```
Training Accuracy: 0.9428571428571428
```

```
Testing Accuracy: 0.9777777777777777
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.92	0.96	13
2	0.93	1.00	0.96	13
accuracy		0.98		45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

PRACTICAL 7(B)

AIM: Train a Naive Bayes' model using a given dataset and calculate class probabilities.

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report


# Step 1: Load dataset

iris = load_iris()

X, y = iris.data, iris.target


# Step 2: Split into train/test

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 3: Initialize Naive Bayes classifier

nb_clf = GaussianNB()


# Step 4: Train model

nb_clf.fit(X_train, y_train)


# Step 5: Predict class labels
```

```

y_pred = nb_clf.predict(X_test)

# Step 6: Predict class probabilities
y_prob = nb_clf.predict_proba(X_test)

# Step 7: Evaluate
print("Testing Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Show first 5 predictions with probabilities
for i in range(5):
    print(f"Sample {i+1}: Predicted={y_pred[i]}, Probabilities={y_prob[i]}")

```

OUTPUT:

```

Testing Accuracy: 0.9777777777777777

Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00        19
     1       1.00      0.92      0.96        13
     2       0.93      1.00      0.96        13

 accuracy              0.98        45
 macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45

Sample 1: Predicted=1, Probabilities=[4.15880005e-88 9.95527834e-01 4.47216606e-03]
Sample 2: Predicted=0, Probabilities=[1.00000000e+00 1.31031235e-13 2.21772205e-20]
Sample 3: Predicted=2, Probabilities=[9.83170191e-285 2.70138564e-012 1.00000000e+000]
Sample 4: Predicted=1, Probabilities=[9.54745274e-92 9.74861431e-01 2.51385686e-02]
Sample 5: Predicted=1, Probabilities=[1.0867956e-103 8.3191070e-001 1.6808930e-001]

```

PRACTICAL 7(C)

AIM: Evaluate the accuracy of the model on test data and analyze the results.

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report


# Step 1: Load dataset

iris = load_iris()

X, y = iris.data, iris.target


# Step 2: Split into train/test

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Step 3: Initialize Naive Bayes classifier

nb_clf = GaussianNB()


# Step 4: Train model

nb_clf.fit(X_train, y_train)


# Step 5: Predict labels and probabilities
```

```

y_pred = nb_clf.predict(X_test)
y_prob = nb_clf.predict_proba(X_test)

# Step 6: Evaluate accuracy
print("Testing Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Step 7: Show sample predictions with probabilities
for i in range(5):

    print(f"Sample {i+1}: True={y_test[i]}, Predicted={y_pred[i]},
    Probabilities={y_prob[i]}")

```

OUTPUT:

```

=====
Testing Accuracy: 0.9777777777777777

Classification Report:
      precision    recall  f1-score   support

0       1.00      1.00      1.00        19
1       1.00      0.92      0.96        13
2       0.93      1.00      0.96        13

 accuracy                   0.98         45
macro avg      0.98      0.97      0.97         45
weighted avg   0.98      0.98      0.98         45

Sample 1: True=1, Predicted=1, Probabilities=[4.15880005e-88 9.95527834e-01 4.47216606e-03]
Sample 2: True=0, Predicted=0, Probabilities=[1.00000000e+00 1.31031235e-13 2.21772205e-20]
Sample 3: True=2, Predicted=2, Probabilities=[9.83170191e-285 2.70138564e-012 1.00000000e+00
0]
Sample 4: True=1, Predicted=1, Probabilities=[9.54745274e-92 9.74861431e-01 2.51385686e-02]
Sample 5: True=1, Predicted=1, Probabilities=[1.0867956e-103 8.3191070e-001 1.6808930e-001]
>|

```

PRACTICAL 8(A)

AIM: Implement the K-NN algorithm for classification or regression.

- **K-NN for Classification (Iris dataset)**

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report


# Load dataset

iris = load_iris()

X, y = iris.data, iris.target


# Split into train/test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Initialize K-NN classifier (k=5)

knn_clf = KNeighborsClassifier(n_neighbors=5)

knn_clf.fit(X_train, y_train)


# Predictions

y_pred = knn_clf.predict(X_test)


# Evaluation
```

```
print("Classification Accuracy:", accuracy_score(y_test, y_pred))  
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

```
Classification Accuracy: 1.0  
  
Classification Report:  
      precision    recall  f1-score   support  
  
0       1.00      1.00      1.00       19  
1       1.00      1.00      1.00       13  
2       1.00      1.00      1.00       13  
  
accuracy                1.00      45  
macro avg       1.00      1.00      1.00      45  
weighted avg    1.00      1.00      1.00      45
```

- **K-NN for Regression (Diabetes dataset)**

CODE:

```
import numpy as np  
  
from sklearn.datasets import load_diabetes  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.neighbors import KNeighborsRegressor  
  
from sklearn.metrics import mean_squared_error, r2_score  
  
  
# Load dataset  
  
data = load_diabetes()  
  
X, y = data.data, data.target  
  
  
# Split into train/test  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)  
  
  
# Initialize K-NN regressor (k=5)
```

```
knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X_train, y_train)

# Predictions
y_pred = knn_reg.predict(X_test)

# Evaluation
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

OUTPUT:

```
| Mean Squared Error: 3222.117894736842
| R2 Score: 0.4031244536507893
|
```

PRACTICAL 8(B)

AIM: Apply the K-NN algorithm to a given dataset and predict the class or value for test data.

- **K-NN Classification (Iris dataset)**

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report


# Load dataset

iris = load_iris()

X, y = iris.data, iris.target


# Split into train/test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Initialize K-NN classifier (k=5)

knn_clf = KNeighborsClassifier(n_neighbors=5)

knn_clf.fit(X_train, y_train)


# Predict on test data

y_pred = knn_clf.predict(X_test)
```

```
# Evaluate

print("Classification Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Show first 5 predictions

print("First 5 Predictions:", y_pred[:5])
```

OUTPUT:

```
Classification Accuracy: 1.0

Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00        19
     1       1.00      1.00      1.00        13
     2       1.00      1.00      1.00        13

 accuracy                1.00         45
 macro avg       1.00      1.00      1.00         45
weighted avg       1.00      1.00      1.00         45

First 5 Predictions: [1 0 2 1 1]
```

- **K-NN Regression (Diabetes dataset)**

CODE:

```
import numpy as np

from sklearn.datasets import load_diabetes

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import mean_squared_error, r2_score

# Load dataset

data = load_diabetes()

X, y = data.data, data.target
```

```
# Split into train/test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Initialize K-NN regressor (k=5)

knn_reg = KNeighborsRegressor(n_neighbors=5)

knn_reg.fit(X_train, y_train)


# Predict on test data

y_pred = knn_reg.predict(X_test)


# Evaluate

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

print("R2 Score:", r2_score(y_test, y_pred))


# Show first 5 predictions

print("First 5 Predictions:", y_pred[:5])
```

OUTPUT:

```
Mean Squared Error: 3222.117894736842
R2 Score: 0.4031244536507893
First 5 Predictions: [125.6 160.2 153. 238. 153.4]
```

PRACTICAL 8(C)

AIM: Evaluate the accuracy or error of the predictions and analyze the results.

CODE:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Load dataset

iris = load_iris()

X, y = iris.data, iris.target


# Split into train/test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Initialize K-NN classifier (k=5)

knn_clf = KNeighborsClassifier(n_neighbors=5)

knn_clf.fit(X_train, y_train)


# Predict on test data

y_pred = knn_clf.predict(X_test)


# Evaluate accuracy and error
```

```

accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print("Classification Accuracy:", accuracy)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", conf_matrix)

```

OUTPUT:

Classification Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Confusion Matrix:

```

[[19 0 0]
 [ 0 13 0]
 [ 0 0 13]]

```

PRACTICAL 9(A)

AIM: Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent itemsets.

CODE:

```
import pandas as pd

from mlxtend.frequent_patterns import apriori, association_rules

# Step 1: Create a sample dataset (market basket transactions)
dataset = [
    ['milk', 'bread', 'butter'],
    ['bread', 'butter'],
    ['milk', 'bread'],
    ['milk', 'bread', 'butter', 'jam'],
    ['bread', 'butter', 'jam'],
    ['milk', 'bread', 'jam']
]

# Step 2: Convert dataset into one-hot encoded DataFrame
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()

te_ary = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_ary, columns=te.columns_)

print("Transaction DataFrame:\n", df)
```

```
# Step 3: Apply Apriori to find frequent itemsets
```

```
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)
```

```
print("\nFrequent Itemsets:\n", frequent_itemsets)
```

```
# Step 4: Generate association rules
```

```
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
```

```
print("\nAssociation Rules:\n", rules[['antecedents','consequents','support','confidence','lift']])
```

OUTPUT:

```
Transaction DataFrame:
```

```
   bread  butter  jam  milk
0  True   True  False  True
1  True   True  False  False
2  True  False  False  True
3  True   True   True  True
4  True   True   True  False
5  True  False   True  True
```

```
Frequent Itemsets:
```

```
   support  itemsets
0  1.000000  frozenset({bread})
1  0.666667  frozenset({butter})
2  0.500000  frozenset({jam})
3  0.666667  frozenset({milk})
4  0.666667  frozenset({butter, bread})
5  0.500000  frozenset({bread, jam})
6  0.666667  frozenset({milk, bread})
```

```
Warning (from warnings module):
```

```
File "C:\Users\dell\AppData\Local\Programs\Python\Python314\Lib\site-packages\mlxtend\frequent_patterns\association_rules.py", line 184
    cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)
```

```
RuntimeWarning: invalid value encountered in divide
```

```
Association Rules:
```

```
   antecedents  consequents  support  confidence  lift
0  frozenset({butter})  frozenset({bread})  0.666667    1.0  1.0
1  frozenset({jam})  frozenset({bread})  0.500000    1.0  1.0
2  frozenset({milk})  frozenset({bread})  0.666667    1.0  1.0
```

PRACTICAL 9(B)

AIM: Generate association rules from the frequent itemsets and calculate their support and confidence.

CODE:

```
import pandas as pd

from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

# Step 1: Create a sample dataset (market basket transactions)
dataset = [
    ['milk', 'bread', 'butter'],
    ['bread', 'butter'],
    ['milk', 'bread'],
    ['milk', 'bread', 'butter', 'jam'],
    ['bread', 'butter', 'jam'],
    ['milk', 'bread', 'jam']
]

# Step 2: Convert dataset into one-hot encoded DataFrame
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

print("Transaction DataFrame:\n", df)
```

Step 3: Apply Apriori to find frequent itemsets

```
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)
```

```
print("\nFrequent Itemsets:\n", frequent_itemsets)
```

Step 4: Generate association rules with support & confidence

```
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
```

```
print("\nAssociation Rules:\n", rules[['antecedents','consequents','support','confidence','lift']])
```

OUTPUT:

```
Transaction DataFrame:
   bread  butter  jam  milk
0  True   True  False  True
1  True   True  False  False
2  True  False  False  True
3  True   True   True  True
4  True   True   True  False
5  True  False   True  True
```

```
Frequent Itemsets:
   support  itemsets
0  1.000000  frozenset({bread})
1  0.666667  frozenset({butter})
2  0.500000  frozenset({jam})
3  0.666667  frozenset({milk})
4  0.666667  frozenset({bread, butter})
5  0.500000  frozenset({jam, bread})
6  0.666667  frozenset({bread, milk})
```

Warning (from warnings module):

File "C:\Users\dell\AppData\Local\Programs\Python\Python314\Lib\site-packages\mlxtend\frequent_patterns\association_rules.py", line 184
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)

RuntimeWarning: invalid value encountered in divide

```
Association Rules:
   antecedents  consequents  support  confidence  lift
0  frozenset({butter})  frozenset({bread})  0.666667    1.0  1.0
1  frozenset({jam})  frozenset({bread})  0.500000    1.0  1.0
2  frozenset({milk})  frozenset({bread})  0.666667    1.0  1.0
```

PRACTICAL 9(C)

AIM: Interpret and analyze the discovered association rules.

CODE:

```
import pandas as pd

from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

# Step 1: Create a sample dataset (market basket transactions)
dataset = [
    ['milk', 'bread', 'butter'],
    ['bread', 'butter'],
    ['milk', 'bread'],
    ['milk', 'bread', 'butter', 'jam'],
    ['bread', 'butter', 'jam'],
    ['milk', 'bread', 'jam']
]

# Step 2: Convert dataset into one-hot encoded DataFrame
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

print("Transaction DataFrame:\n", df)
```

```

# Step 3: Apply Apriori to find frequent itemsets

frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)

print("\nFrequent Itemsets:\n", frequent_itemsets)


# Step 4: Generate association rules

rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

print("\nAssociation Rules:\n", rules[['antecedents','consequents','support','confidence','lift']])


# Step 5: Interpret rules

print("\n--- Interpretation ---")

for idx, row in rules.iterrows():

    antecedent = list(row['antecedents'])[0]

    consequent = list(row['consequents'])[0]

    print(f"Rule: If a customer buys {antecedent}, they also buy {consequent}")

    print(f" - Support: {row['support']:.2f} (fraction of transactions containing both)")

    print(f" - Confidence: {row['confidence']:.2f} (probability of buying {consequent} given {antecedent})")

    print(f" - Lift: {row['lift']:.2f} (strength compared to random chance)\n")

```

OUTPUT:

```
Transaction DataFrame:
  bread butter  jam  milk
0  True   True  False  True
1  True   True  False  False
2  True  False  False  True
3  True   True   True  True
4  True   True   True  False
5  True  False   True  True
```

```
Frequent Itemsets:
support      itemsets
0  1.000000  frozenset({bread})
1  0.666667  frozenset({butter})
2  0.500000  frozenset({jam})
3  0.666667  frozenset({milk})
4  0.666667  frozenset({bread, butter})
5  0.500000  frozenset({bread, jam})
6  0.666667  frozenset({bread, milk})
```

```
Association Rules:
      antecedents  consequents  support  confidence  lift
0  frozenset({butter})  frozenset({bread})  0.666667      1.0  1.0
1   frozenset({jam})  frozenset({bread})  0.500000      1.0  1.0
2   frozenset({milk})  frozenset({bread})  0.666667      1.0  1.0
```

--- Interpretation ---

Rule: If a customer buys butter, they also buy bread

- Support: 0.67 (fraction of transactions containing both)
- Confidence: 1.00 (probability of buying bread given butter)
- Lift: 1.00 (strength compared to random chance)

Rule: If a customer buys jam, they also buy bread

- Support: 0.50 (fraction of transactions containing both)
- Confidence: 1.00 (probability of buying bread given jam)
- Lift: 1.00 (strength compared to random chance)

Rule: If a customer buys milk, they also buy bread

- Support: 0.67 (fraction of transactions containing both)
- Confidence: 1.00 (probability of buying bread given milk)
- Lift: 1.00 (strength compared to random chance)